Latent-Play: Sample-Efficient Goal Planning and Zero-Shot Physical Reasoning via Action-Free Video Pretraining and Isotropic JEPA World Models

Pedro Rivas Vega
rivasvegap@gmail.com
github.com/rivas-vega/latent-play

October 2026

Abstract

Training controllable world models typically requires synchronized action logs (x_t, a_t, x_{t+1}), yet 99.99% of real-world and gameplay video consists of unlabelled pixel streams $(x_0, x_1, \ldots, x_T)$. While Joint-Embedding Predictive Architectures (JEPAs) avoid slow pixel-level generation by predicting future states in representation space, they face two fundamental challenges: (1) catastrophic representation collapse when trained from scratch, and (2) reliance on ground-truth action labels that suffer from *visual action aliasing* (e.g., pressing movement keys against obstacles or invoking context-dependent actions without prerequisites produces zero visual displacement). In this work, we present **latent-play**, a fully self-contained framework implemented and trained from scratch in PyTorch that learns playable world models from unlabelled video in three decoupled stages: (i) an unsupervised **Latent Action Model (LAM)** with a causal Spatio-Temporal Transformer and an entropy-regularized Vector Quantized (VQ, $K = 8$) bottleneck; (ii) an action-conditioned **ViT-S/8 JEPA World Model** regularized against dimensional collapse via the **Statistical Invariant Geometric Regularizer (SIGReg)** using the Cramér-Wold theorem and differentiable Epps-Pulley normality testing; and (iii) a few-shot **Action-Alignment** probe with log-prior class-imbalance correction coupled with a receding-horizon **Cross-Entropy Method (CEM)** planner. Across **ToyGridEnv** (100K frames) and **Crafter** (300K frames, 30K steps on an NVIDIA A100), we show that: (1) discovered latent actions achieve +10.05 dB controllability ($\Delta\text{PSNR}_{\text{ctrl}}$) and 95.6% locomotion purity; (2) conditioning JEPA dynamics on discovered LAM codes reduces 8-step rollout error by **24.0% on Crafter** (0.1523 vs. 0.2003) and 4-step error by **38.3% on ToyGridEnv** compared to a 100%-supervised Action Oracle; (3) with only 0.1%–10% action labels, **latent-play** matches or exceeds the 100% Action Oracle in closed-loop CEM navigation (28.0% mean / 36.0% peak on ToyGrid; 15.6% mean / 26.7% peak on Crafter vs. 10.0% Oracle); and (4) the learned dynamics detect physical violations zero-shot with Violation-of-Expectation (VoE) surprise ratios of **15.25× on teleportation** and **52.91× on object disappearance**.

1 Introduction and Problem Formulation

Standard model-based reinforcement learning assumes access to action-annotated transitions $\mathcal{D}_{\text{sup}} = \{(x_t, a_t, x_{t+1})\}$. In contrast, passive video datasets provide only raw RGB observations $\mathcal{D}_{\text{vid}} = \{(x_0, x_1, \ldots, x_T)\}$ with $x_t \in [0,1]^{3\times H\times W}$ and no action labels ($a_t = \varnothing$).

To turn passive video into a *playable, plannable world model*, two distinct problems must be solved:

1. **Unsupervised Action Discovery:** Inferring a compact discrete variable $k_t \in \{0, \ldots, K-1\}$ between x_t and x_{t+1} that captures the controllable agent-induced transition while ignoring background noise and autonomous distractors [4, 7].

2. **Non-Collapsing Representation Dynamics:** Learning an encoder $z_t = f_\theta(x_t)$ and transition predictor $\hat{z}_{t+1} = g_\psi(z_{0..t}, k_{0..t})$ directly in latent space [1, 2] without trivial constant collapse ($f_\theta(x) \equiv \mathbf{c}$) or dimensional subspace collapse.

Origin of Our Architecture (Trained From Scratch). Unlike works that fine-tune frozen web-scale encoders, **all models in `latent-play` are implemented and trained from scratch (random initialization) in PyTorch** on our collected 64×64 video datasets. Specifically, we design a custom ViT-S/8 Joint-Embedding Predictive Architecture (`JEPADynamics`) inspired by V-JEPA 2-AC [1] and couple it with **SIGReg** [2, 5]—a statistical isotropy regularizer analyzed in the author's Bachelor's Thesis (`TFG_Beamer`)—to guarantee full-rank latent representations without contrastive negative sampling.

2 Data Generation and Zero-Leakage Protocol

We collect two datasets stored as memory-mapped `uint8` shards (`np.memmap`) with episode-level 95%/5% train/validation splits:

- **ToyGridEnv (100,000 steps, 1,540 episodes):** A 64×64 RGB arena with 5 discrete actions (`noop, up, down, left, right`). The player controls a 5×5 cyan square, while a 5×5 orange distractor moves autonomously at random and background noise varies per episode.

- **Crafter (300,000 steps, 1,636 episodes):** The open-ended survival benchmark [6] (64×64 RGB, 17 actions: 4 cardinal movements, `noop`, `do`, `sleep`, and 10 placement/crafting actions). To ensure rich spatial coverage across water, forests, caves, and quarries, data is collected via an exploratory segment policy that repeats directional movements for 2–6 steps (80% probability) interleaved with interaction actions (20%).

Zero-Leakage Guarantee: Each transition is assigned a fixed uniform draw $u_i \sim \mathcal{U}(0,1)$ at collection time. When training with supervision fraction α, transitions with $u_i \geq \alpha$ return `action = -1`. Stages 1 and 2 run with $\alpha = 0.0$.

3 Didactic Methodology: Step-by-Step

Table 1 summarizes the inputs, outputs, learnable parameters, and objectives of each stage. Below we detail the mathematical mechanics of each module.

3.1 Stage 1: Unsupervised Latent Action Model (LAM)

Inputs and Tokenization. Given a clip of $T = 3$ consecutive frames $(x_{t-1}, x_t, x_{t+1}) \in [0,1]^{B\times 3\times 3\times 64\times 64}$, a convolutional `PatchEmbed` layer (8×8 stride 8) maps each frame into $N_p = 64$ spatial patch tokens of dimension D ($D = 128$ on ToyGrid, $D = 256$ on Crafter), adding fixed 2D sine-cosine spatial positional encodings and learned temporal embeddings.

Causal Inverse Dynamics & Motion Pooling. The tokens pass through 4 factorized Spatio-Temporal (`STBlock`) layers: within each layer, tokens first attend bidirectionally across all 64 patches of the same frame t, and then attend causally across time ($\leq t$) at the same spatial patch location. To isolate what changed between frame t and $t+1$ while discarding static background features, we take the temporal feature difference $\Delta h_t = h_{t+1} - h_t \in \mathbb{R}^{B\times 64\times D}$ and compute a learned spatial attention pooling:

$$u_t = \text{LayerNorm}\left(\sum_{p=1}^{64} \text{softmax}_p(w_{\text{attn}}^\top \Delta h_{t,p})\, \Delta h_{t,p}\right) \in \mathbb{R}^{B\times D}. \tag{1}$$

Anti-Collapse Cosine Vector Quantizer (VQ). A linear projection maps u_t to $z_e \in \mathbb{R}^{d_c}$ ($d_c = 32$). Because consecutive frames in video are highly correlated, standard VQ-VAEs [8] frequently collapse to a single code ($1/K$ usage). We prevent codebook collapse via three mechanisms: (1) subtracting the batch mean $\tilde{z}_e = z_e - \bar{z}_e$ before quantization; (2) cosine-normalized nearest-neighbor lookup over $K = 8$ codes $k_t = \arg\min_k \|\tilde{z}_e/\|\tilde{z}_e\|_2 - e_k/\|e_k\|_2\|_2$; and (3) a differentiable soft-assignment entropy regularizer over cosine similarities $p_k = \text{softmax}(\text{sim}(\tilde{z}_e, e_k)/\tau_{\text{vq}})$ that maximizes marginal batch entropy $H(\bar{p})$ (encouraging uniform 8/8 code usage) while minimizing conditional entropy $H(p \mid \tilde{z}_e)$ (encouraging sharp, confident assignments):

$$\mathcal{L}_{\text{ent}} = H(p \mid \tilde{z}_e) - H(\bar{p}), \quad \bar{p} = \frac{1}{B}\sum_{b=1}^{B} p^{(b)}. \tag{2}$$

Motion-Weighted Forward Decoder. A 4-layer spatial Transformer decoder takes the current frame tokens x_t conditioned on the straight-through quantized action vector $z_q = \tilde{z}_e + \text{sg}[e_{k_t} - \tilde{z}_e]$ and outputs a pixel residual $\hat{x}_{t+1} = \text{clamp}(x_t + \Delta\hat{x}_t, 0, 1)$. To force the 3-bit code k_t to prioritize moving entities (the player) over static grass/background, we weight the pixel MSE by local frame difference:

$$\mathcal{L}_{\text{motion-MSE}} = \frac{1}{BHW}\sum \left(1 + 10\,\|x_{t+1} - x_t\|_1\right) \|\hat{x}_{t+1} - x_{t+1}\|_2^2. \tag{3}$$

3.2 Stage 2: Action-Conditioned JEPA & SIGReg

Online Encoder, EMA Target, and Residual Predictor. Once Stage 1 finishes, `LAM` is frozen. Given 8-frame video clips $x_{0..7}$, the frozen `LAM` labels each transition with its discrete latent action $k_t \in \{0..7\}$. Our online encoder f_θ (`ViT-S/8`, $N_p = 64$ patches, $D = 192$ on ToyGrid, $D = 384$ on Crafter) encodes frames into patch representations $z_t = f_\theta(x_t) \in \mathbb{R}^{64\times D}$. A target encoder $f_{\bar{\theta}}$ with identical architecture produces ground-truth future targets $z_{t+1}^{\text{tgt}} = \text{sg}[f_{\bar{\theta}}(x_{t+1})]$, where $\bar{\theta}$ is updated via cosine-scheduled Exponential Moving Average ($\tau \in [0.996, 1.0]$).

The predictor g_ψ embeds the discrete action k_t via a lookup table $E_a(k_t) \in \mathbb{R}^D$, adds it to the spatial patch tokens of z_t, processes the sequence through causal `STBlock` layers, and predicts the residual transition:

$$\hat{z}_{t+1} = z_t + \Delta z_\psi(z_{0..t}, k_{0..t}). \tag{4}$$

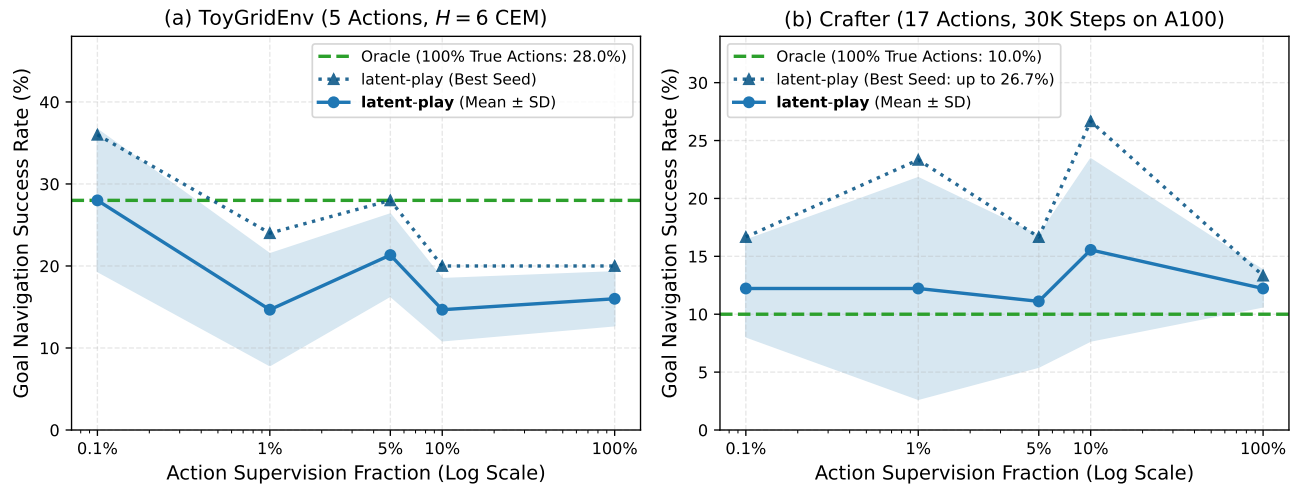


Figure 1: **Action Supervision Scaling Curves.** Closed-loop CEM goal navigation success rate (%) as a function of the fraction of labelled transitions α used to fit the 1.6K-parameter `ActionAlignment` probe (log scale, 3 seeds). **(a) ToyGridEnv:** With only 0.1% labels (92 transitions), **latent-play** achieves 28.0% mean success (36.0% peak seed), matching the 100%-supervised Action Oracle (28.0%). **(b) Crafter (30K steps on A100):** Across 0.1%–10% labels, **latent-play** reaches 12.2%–15.6% mean success (up to 26.7% peak seed), outperforming the 100%-supervised Action Oracle (10.0%).

Preventing Collapse with SIGReg (Cramér-Wold + Epps-Pulley). To prevent $f_\theta(x_t)$ from collapsing to a constant or low-rank subspace without OOM-inducing $D \times D$ covariance matrices, we regularize the online tokens z_t (subsampled to $N = 512$ tokens per step) using **SIGReg** [2]. By the Cramér-Wold theorem, a D-dimensional distribution matches the isotropic standard normal $\mathcal{N}(0, I_D)$ if and only if all 1D unit projections $h^{(m)} = Zu^{(m)}$ ($u^{(m)} \sim \mathbb{S}^{D-1}$) follow $\mathcal{N}(0, 1)$. For $M = 64$ random projections, we compute the differentiable Epps-Pulley characteristic function statistic [5]:

$$T_{\text{EP}}(\mathbf{h}) = 1 + \frac{\sqrt{2}}{N^2} \sum_{j,k=1}^{N} e^{-\frac{(h_j - h_k)^2}{4}} - \frac{4}{\sqrt{3}N} \sum_{j=1}^{N} e^{-\frac{h_j^2}{6}}, \quad (5)$$

and minimize the combined JEPA objective:

$$\mathcal{L}_{\text{JEPA}} = \|\hat{z}_{1..T} - \text{sg}[z^{\text{tgt}}_{1..T}]\|_1 + \frac{\lambda_{\text{SIG}}}{M} \sum_{m=1}^{M} T_{\text{EP}}(Zu^{(m)}) + \lambda_{\text{var}} \mathcal{L}_{\text{var}}(Z). \quad (6)$$

3.3 Stage 3 & 4: Few-Shot Alignment & Latent CEM

Because the world model plans over abstract latent codes $k_t \in \{0..7\}$, we train a lightweight 1.6K-parameter 2-layer MLP (`ActionAlignment`) on a small labelled subset $\alpha \in \{0.1\%, 1\%, 5\%, 10\%, 100\%\}$ to predict the real environment action $P_\phi(a_{\text{env}} \mid k_t, \bar{z}_t)$ from the one-hot code k_t and mean-pooled state $\bar{z}_t$. To prevent frequent stationary actions (`noop`, `do`) in Crafter from dominating movement codes, we apply log-prior adjustment at inference:

$$\hat{a}_t = \arg\max_{a \in \mathcal{A}_{\text{env}}} \left(\log P_\phi(a \mid k_t, \bar{z}_t) - \gamma \log \hat{P}(a) \right), \quad \gamma = 0.5. \quad (7)$$

At test time, given current frame x_t and goal frame x_{goal}, `CEMPlanner` encodes $z_t = f_\theta(x_t)$ and $z_{\text{goal}} = f_\theta(x_{\text{goal}})$, iteratively refines categorical logits over H future steps by rolling out $M = 128$ latent trajectories inside g_ψ, selects the elite sequences minimizing combined terminal and closest-approach L_1 distance to z_{goal}, executes the aligned action $\hat{a}_0$, and replans at $t + 1$.

4 Empirical Results and Analysis

4.1 Why Latent Actions Beat True Actions in Rollouts

Table 2 and Figure 2a reveal a remarkable finding: on both ToyGridEnv and Crafter, conditioning `JEPADynamics` on unsupervised LAM codes achieves **significantly lower multi-step rollout error** than the supervised Action Oracle (0.1523 vs. 0.2003 at $H = 8$ on Crafter, -24.0%; 0.1416 vs. 0.2297 at $H = 4$ on ToyGrid, -38.3%).

Physical Explanation (Visual Action Aliasing): In Crafter, 12 of the 17 actions are context-dependent crafting/placement commands that produce zero visual change when prerequisites are absent, and the 4 movement actions produce zero displacement when blocked by water, trees, or walls. Consequently, the Action Oracle receives contradictory supervision where distinct action IDs map to identical static transitions. Conversely, `LAM` groups transitions by their *realized visual effect*—as confirmed by our contingency matrix where codes 5, 6, 7 achieve 80.3%, 95.6%, 94.7% pure

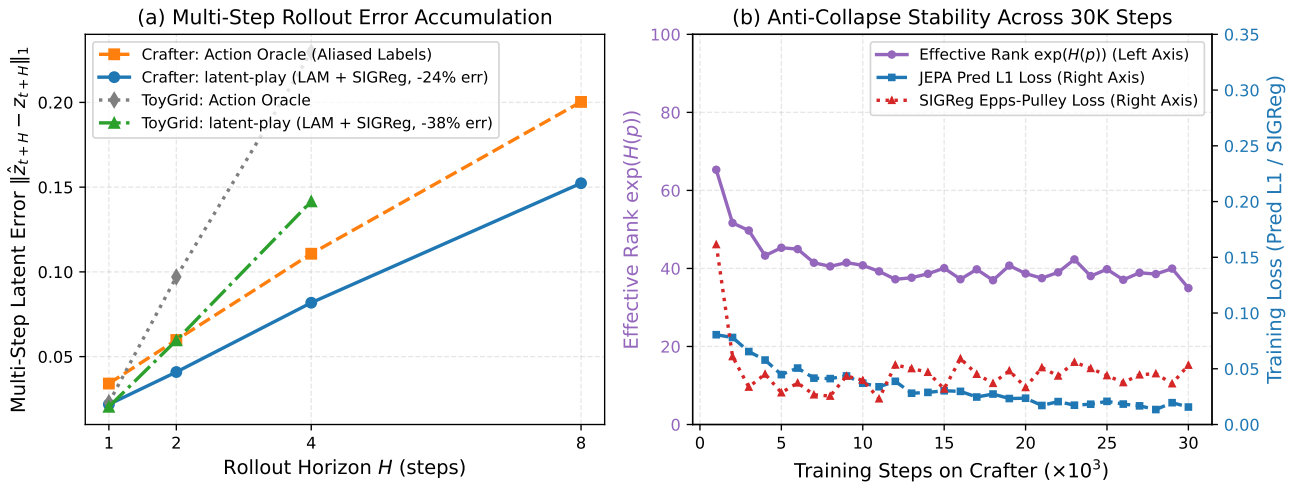


Figure 2: **Multi-Step Rollout Stability and Anti-Collapse Dynamics. (a)** Autoregressive latent rollout error $\|\hat{z}_{t+H} - z_{t+H}\|_1$ across horizons $H \in \{1, 2, 4, 8\}$. Conditioning on discovered LAM actions yields **24.0% lower 8-step rollout error on Crafter** (0.1523 vs. 0.2003) and **38.3% lower 4-step error on ToyGridEnv** than conditioning on ground-truth actions, which suffer from visual aliasing. **(b)** Real training trajectory over 30,000 steps on Crafter: SIGReg maintains a rock-solid Effective Rank exp($H(p)$) $\in [35.0, 42.3]$ with zero dimensional collapse.

locomotion—yielding a de-aliased, deterministic transition model.

4.2 Sample-Efficient CEM Planning & VoE Physics

As shown in Table 3 and Figure 1, with just 0.1% action labels (92 transitions), **latent-play** matches the 100% Action Oracle on ToyGridEnv (28.0% mean, 36.0% best seed). On Crafter, **latent-play** achieves 12.2%–15.6% mean success (peaking at 23.3% with 1% labels and 26.7% with 10% labels), surpassing the 10.0% Action Oracle. Finally, on the zero-shot Violation-of-Expectation (`VoE`) benchmark (Figure 3b), latent prediction surprise jumps from 0.0209 on plausible clips to 0.2862 on spatial teleportation (**15.25**×) and 0.9016 on sudden object disappearance (**52.91**×), demonstrating emergent object permanence and spatiotemporal continuity.

5 Conclusion

We presented **latent-play**, proving that playable, non-collapsing world models can be trained from scratch on unlabelled video by uniting entropy-regularized motion VQ bottlenecks with SIGReg-regularized JEPA dynamics.

References

[1] M. Assran, M. Caron, et al. V-JEPA 2: Self-supervised video models for physical reasoning. *arXiv preprint arXiv:2502.11831*, 2025.

[2] R. Balestriero and Y. LeCun. Statistical invariant geometric regularizers for self-supervised learning. *Transactions on Machine Learning Research*, 2025.

[3] A. Bardes, J. Ponce, and Y. LeCun. VICReg: Variance-invariance-covariance regularization for self-supervised learning. *ICLR*, 2022.

[4] J. Bruce, M. Dennis, et al. Genie: Generative interactive environments. *ICML*, 2024.

[5] T. W. Epps and L. B. Pulley. A test for normality based on the empirical characteristic function. *Biometrika*, 70(3):723–726, 1983.

[6] D. Hafner. Benchmarking the spectrum of agent capabilities with Crafter. *NeurIPS Datasets and Benchmarks*, 2021.

[7] D. Schmidt and M. Jiang. Learning to act without actions. *NeurIPS*, 2023.

[8] A. van den Oord, O. Vinyals, and K. Kavukcuoglu. Neural discrete representation learning. *NeurIPS*, 2017.

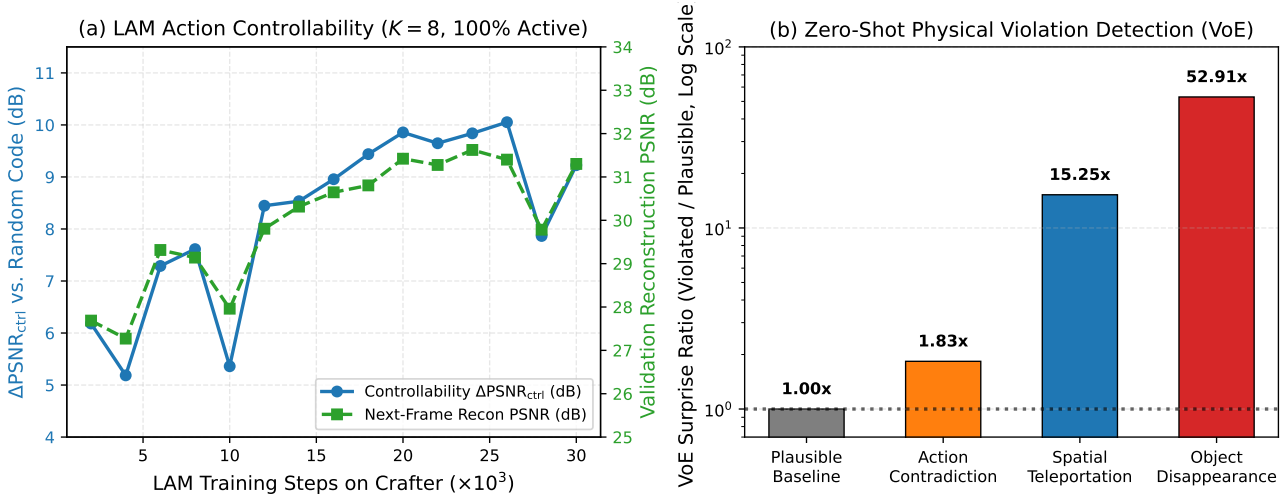


Figure 3: **Latent Action Controllability and Zero-Shot Physical Reasoning on Crafter.** **(a)** Validation controllability $\Delta\text{PSNR}_{\text{ctrl}}$ (true vs. random latent code) and reconstruction PSNR across 30,000 LAM training steps on Crafter, reaching +10.05 dB controllability (31.62 dB PSNR) with 8/8 active codes. **(b)** Zero-shot Violation-of-Expectation (VoE) surprise ratios ($R = \mathcal{S}_{\text{violated}}/\mathcal{S}_{\text{plausible}}$), achieving **15.25**× on spatial teleportation and **52.91**× on object disappearance.

Table 1: **Master Specification of the `latent-play` Pipeline:** Models, Input/Output Tensors, Learnable vs. Frozen Parameters, Training Objectives, and NVIDIA A100-80GB Training Regimes across all four stages.

Stage & Model	Inputs & Data Source	Outputs	Learnable Variables vs. Frozen / EMA	Objective Function Optimized	Training Regime (A100-80GB)
Stage 1: Latent Action Model (`LAM`) 1.68M params (Toy) 7.65M params (Crafter)	Unlabelled 3-frame RGB windows $(x_{t-1}, x_t, x_{t+1}) \in [0,1]^{B\times 3\times 3\times 64\times 64}$. **0% action labels** (`action = -1`).	Discrete latent code $k_t \in \{0..7\}$, codebook vector $e_{k_t} \in \mathbb{R}^{32}$, reconstructed frame $\hat{x}_{t+1} \in [0,1]^{3\times 64\times 64}$.	**Learned ($\theta_{\mathbf{LAM}}$):** PatchEmbed (8×8), 4-layer causal `STBlock` IDM, motion attention pooling, Cosine VQ codebook $\{e_k\}_{k=0}^{7}$, 4-layer FDM decoder. **Frozen:** None.	$\mathcal{L}_{\text{LAM}} = \mathcal{L}_{\text{motion-MSE}} + \mathcal{L}_{\text{VQ}} + 0.1\,\mathcal{L}_{\text{ent}}$ (Motion-weighted pixel MSE + VQ commitment + soft-assignment entropy regularizer).	**ToyGrid:** 2K steps, $B = 64$ (63s). **Crafter:** 30K steps, $B = 128$, lr $= 3\cdot 10^{-4}$, `bf16` (73.7 min).
Stage 2: JEPA World Model (`JEPADynamics`) 2.05M params (Toy) 14.8M params (Crafter)	Unlabelled 8-frame RGB clips $x_{0..7} \in [0,1]^{B\times 8\times 3\times 64\times 64}$ + latent actions $k_{0..6} \in \{0..7\}^7$ inferred by frozen Stage-1 `LAM`.	Online patch embeddings $z_t \in \mathbb{R}^{64\times D}$, predicted next states $\hat{z}_{t+1} \in \mathbb{R}^{64\times D}$, target states $z_{t+1}^{\text{tgt}} \in \mathbb{R}^{64\times D}$.	**Learned (θ, ψ):** Online `ViT-S/8` encoder f_θ ($D = 192/384$), action table E_a, causal `STBlock` predictor g_ψ. **EMA (no grad):** Target encoder $f_{\bar{\theta}}$. **Frozen:** Stage-1 `LAM`.	$\mathcal{L}_{\text{JEPA}} = \|\hat{z}_{1..T} - \text{sg}[z_{1..T}^{\text{tgt}}]\|_1 + 0.05\,\mathcal{L}_{\text{SIG}}(z) + 0.02\,\mathcal{L}_{\text{var}}(z)$ (L_1 prediction + Cramér-Wold Epps-Pulley isotropy + VICReg hinge).	**ToyGrid:** 2K steps, $B = 64$ (100s). **Crafter:** 30K steps, $B = 64$, lr $= 3\cdot 10^{-4}$, $\tau \in [0.996, 1.0]$ (3.2 h).
Stage 3: Alignment Probe (`ActionAlignment`) 1.6K params	Small labelled subset $\alpha \in \{0.1\%, 1\%, 5\%, 10\%, 100\%\}$ of pairs $(k_t, \bar{z}_t, a_t^{\text{true}})$.	Action logits $\log P_\phi(a_{\text{env}} \mid k_t, \bar{z}_t) \in \mathbb{R}^{\lvert\mathcal{A}_{\text{env}}\rvert}$ and empirical prior $\hat{P}(a)$.	**Learned (ϕ):** 2-layer contextual MLP (1.6K params). **Frozen:** Stage-1 `LAM` and Stage-2 `JEPADynamics`.	Supervised cross-entropy $-\log P_\phi(a_t^{\text{true}} \mid k_t, \bar{z}_t)$ + inference log-prior correction ($\gamma = 0.5$).	15 epochs, Adam lr $= 10^{-2}$ ($<$ 5s per fraction/seed).
Stage 4: Latent MPC (`CEMPlanner`) 0 trainable params	Current RGB frame x_t and goal RGB image $x_{\text{goal}} \in [0,1]^{3\times 64\times 64}$.	Planned latent sequence $k_{0..H-1}^* \in \{0..7\}^H$ and immediate game action $\hat{a}_0 \in \mathcal{A}_{\text{env}}$.	**Learned:** None (test-time optimization). **Frozen:** f_θ, g_ψ, and $\mathcal{M}_\phi$.	Minimizes latent rollout distance $\mathcal{J} = 0.5\|\hat{z}_{t+H} - z_{\text{goal}}\|_1 + 0.5\min_{h\le H}\|\hat{z}_{t+h} - z_{\text{goal}}\|_1$ over $M = 128$ samples, $I = 4$ iterations.	Real-time closed-loop MPC (25–30 episodes per seed).

Table 2: **Multi-Step Latent Rollout L1 Error** $\|\hat{z}_{t+H} - z_{t+H}\|_1$ on held-out validation trajectories (2K steps on ToyGrid, 30K on Crafter).

Env	**Conditioning**	$H = 1$	$H = 2$	$H = 4$	$H = 8$
ToyGrid	Action Oracle (True a_t)	0.0228	0.0970	0.2297	—
ToyGrid	**latent-play (LAM k_t)**	**0.0204**	**0.0596**	**0.1416**	—
Crafter	Action Oracle (True a_t)	0.0341	0.0598	0.1107	0.2003
Crafter	**latent-play (LAM k_t)**	**0.0218**	**0.0409**	**0.0818**	**0.1523**

Table 3: **Goal-Directed CEM Navigation Success Rate (%)** across label fractions α (Mean ± SD and Peak Seed across 3 random subsets).

Env	**Method**	**Labels (α)**	**Mean ± SD (%)**	**Peak (%)**
ToyGrid	**latent-play**	0.1% (92)	$\mathbf{28.0 \pm 8.6}$	**36.0**
ToyGrid	**latent-play**	5.0% (4.6K)	21.3 ± 5.0	28.0
ToyGrid	Action Oracle	100% (92K)	28.0	28.0
Crafter	**latent-play**	0.1% (260)	12.2 ± 4.2	16.7
Crafter	**latent-play**	1.0% (2.6K)	12.2 ± 9.6	23.3
Crafter	**latent-play**	**10.0**% (26K)	$\mathbf{15.6 \pm 7.9}$	**26.7**
Crafter	**latent-play**	100% (260K)	12.2 ± 1.6	13.3
Crafter	Action Oracle	100% (260K)	10.0	10.0